Name: 張哲語　Student ID: 109062336

3.1

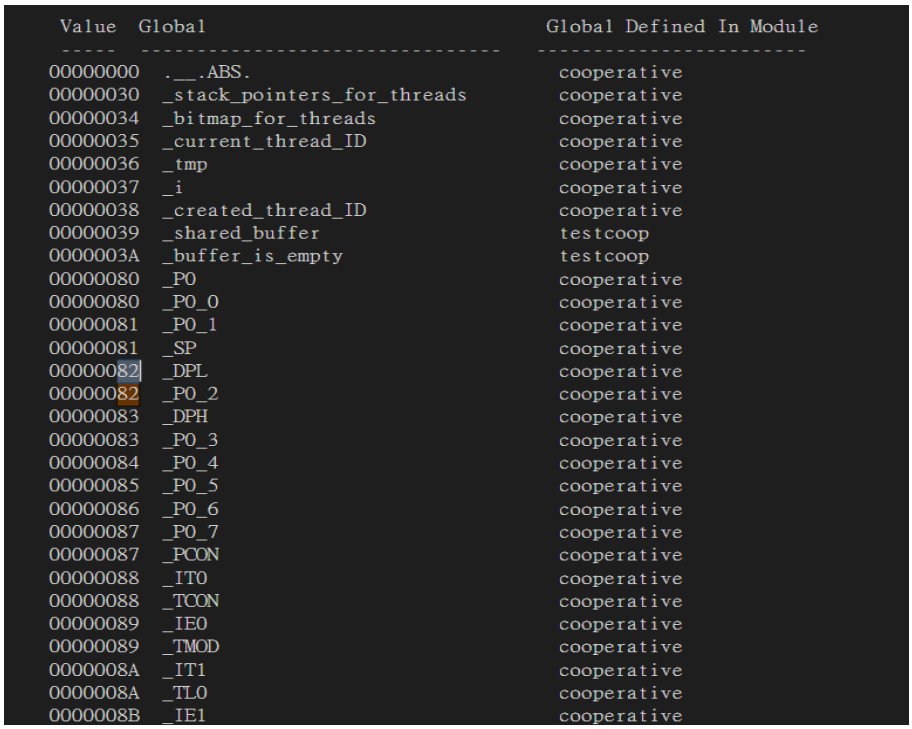
```
user@LAPTOP-REEAC2HM ~/project1
$ make clean
rm *.hex *.ihx *.lnk *.lst *.map *.mem *.rel *.rst *.sym
rm: cannot remove '*.ihx': No such file or directory
rm: cannot remove '*.lnk': No such file or directory
make: *** [Makefile:25: clean] Error 1

user@LAPTOP-REEAC2HM ~/project1
$ ls
Makefile  cooperative.asm  cooperative.c  cooperative.h  testcoop.asm  testcoop.c  testcoop.lk

user@LAPTOP-REEAC2HM ~/project1
$ make
sdcc -c  testcoop.c
testcoop.c:56: warning 158: overflow in implicit constant conversion
sdcc -c  cooperative.c
cooperative.c:199: warning 85: in function ThreadCreate unreferenced function argument : 'fp'
sdcc  -o testcoop.hex testcoop.rel cooperative.rel

user@LAPTOP-REEAC2HM ~/project1
$ ls
Makefile         cooperative.h    cooperative.rst  testcoop.c    testcoop.lst  testcoop.rel
cooperative.asm  cooperative.lst  cooperative.sym  testcoop.hex  testcoop.map  testcoop.rst
cooperative.c    cooperative.rel  testcoop.asm     testcoop.lk   testcoop.mem  testcoop.sym

user@LAPTOP-REEAC2HM ~/project1
$
```

After compiling, files such as testcoop.map, testcoop.hex… has been generated.

3.2

Some addresses of the functions and variables:

```
      Value  Global                          Global Defined In Module
      -----  -------------------------------  -----------------------
C:    00000009  _Producer                      testcoop
C:    0000002F  _Consumer                      testcoop
C:    00000051  _main                          testcoop
C:    00000060  __sdcc_gsinit_startup          testcoop
C:    00000064  __mcs51_genRAMCLEAR            testcoop
C:    00000065  __mcs51_genXINIT               testcoop
C:    00000066  __mcs51_genXRAMCLEAR           testcoop
C:    00000067  _Bootstrap                     cooperative
C:    0000009C  _ThreadCreate                  cooperative
C:    0000012B  _ThreadYield                   cooperative
C:    00000189  _ThreadExit                    cooperative
```

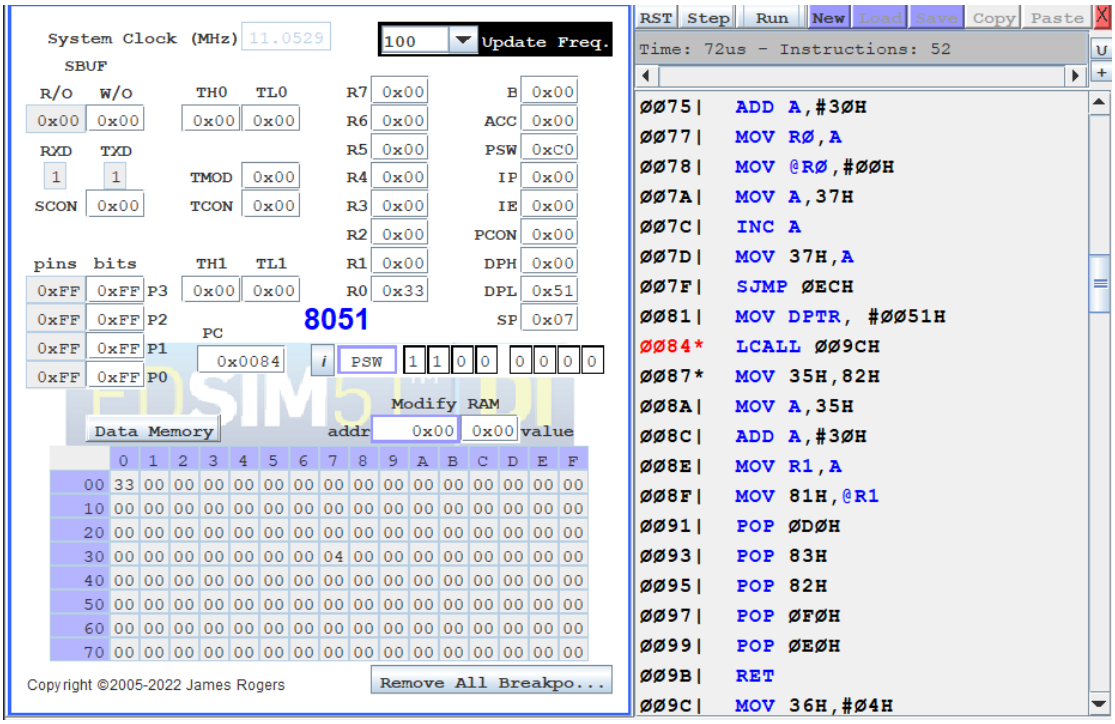


```
 Value  Global                          Global Defined In Module
 -----  -------------------------------  ------------------------
00000000  .__.ABS.                        cooperative
00000030  _stack_pointers_for_threads     cooperative
00000034  _bitmap_for_threads             cooperative
00000035  _current_thread_ID              cooperative
00000036  _tmp                            cooperative
00000037  _i                              cooperative
00000038  _created_thread_ID              cooperative
00000039  _shared_buffer                  testcoop
0000003A  _buffer_is_empty                testcoop
00000080  _P0                             cooperative
00000080  _P0_0                           cooperative
00000081  _P0_1                           cooperative
00000081  _SP                             cooperative
00000082  _DPL                            cooperative
00000082  _P0_2                           cooperative
00000083  _DPH                            cooperative
00000083  _P0_3                           cooperative
00000084  _P0_4                           cooperative
00000085  _P0_5                           cooperative
00000086  _P0_6                           cooperative
00000087  _P0_7                           cooperative
00000087  _PCON                           cooperative
00000088  _IT0                            cooperative
00000088  _TCON                           cooperative
00000089  _IE0                            cooperative
00000089  _TMOD                           cooperative
0000008A  _IT1                            cooperative
0000008A  _TL0                            cooperative
0000008B  _IE1                            cooperative
```

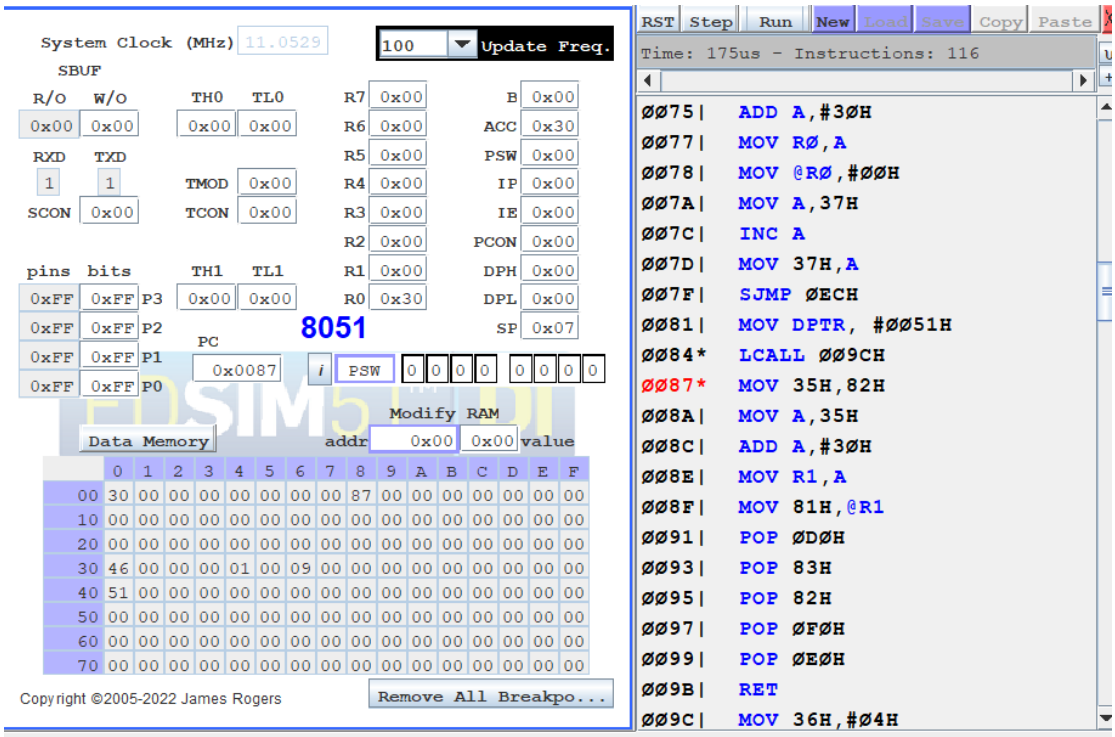
Address of ThreadCreate(): 0x9C

1.

Before the function call of ThreadCreate(main) in Bootstrap(). 0x84



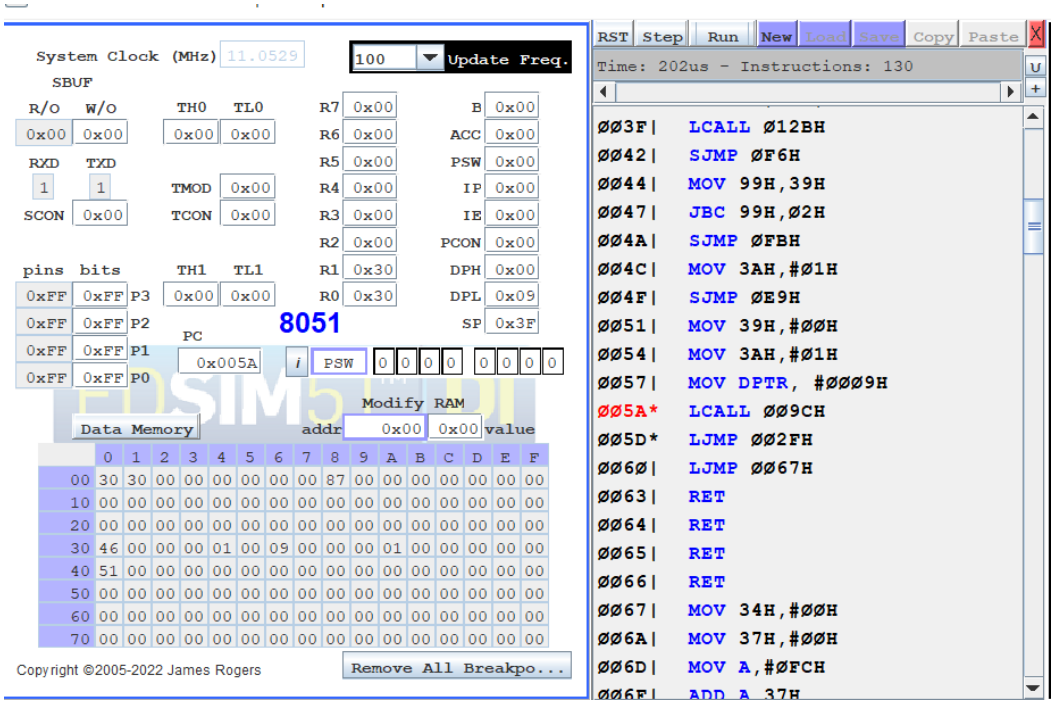
After returning from the ThreadCreate function. 0x87



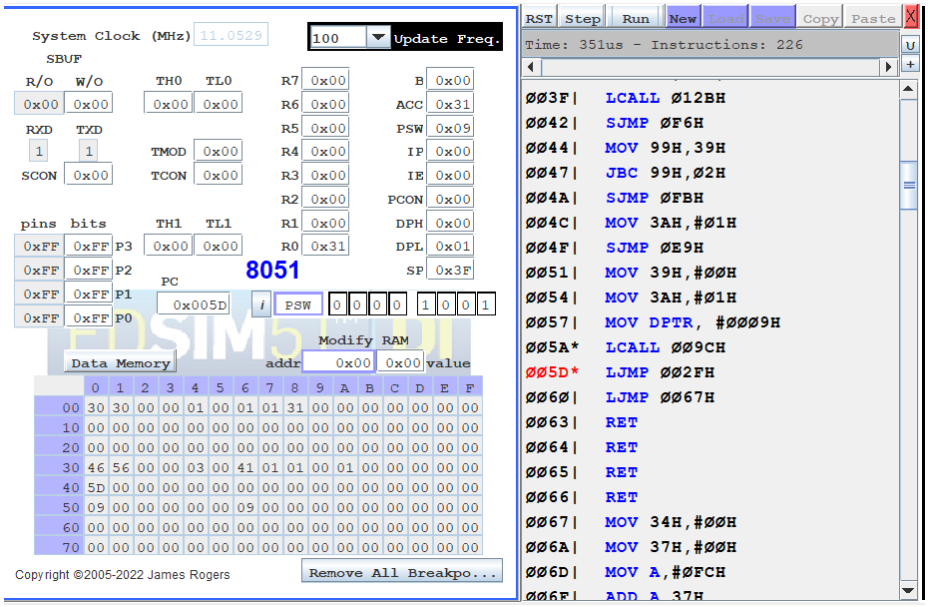
By the screenshot, we can find out that thread 0 has been created since the bitmap for threads(0x34) has been changed to 1. So in terms of the stack, we could check from address 0x40. Moreover, in 0x30, the value of stack_pointers_for_threads[0] is 0x46(address of the top of the stack for thread 1). So we could know that, during the execution of CreateThread(main), SP has been moved to 0x3F, then 7 variables has

been pushed into the stack. DPL, DPH (return addresses to resume the thread), 4 zeros for ACC, B, DPL, DPH, and PSW has been pushed in the stack in order, which corresponds to the values in 0x40, 0x41, 0x42, 0x43, 0x44, 0x45, 0x46. That's why the stack_pointers_for_threads[0] has the value 0x46.

Before the function call of ThreadCreate(Producer) in main(). 0x5A



After returning from the ThreadCreate function. 0x5D



bitmap_for_threads has been updated to 3, means that thread 1 has been created. Similar to the ThreadCreate(main) function explained above, stack_pointers_for_threads[1] gets the value 0x56(address of the top of the stack for thread 1), means that 7 variables has been pushed into the stack for thread 1. DPL, DPH, 4 zeros for ACC, B, DPL, DPH, and PSW are in address 0x50, 0x51, 0x52, 0x53, 0x54, 0x55, 0x56 respectively.

2. We could know that Producer() is running for several reasons:

```
      Value  Global                          Global Defined In Module
      -----  -------------------------------  ------------------------
C:    00000009  _Producer                       testcoop
C:    0000002F  _Consumer                       testcoop
C:    00000051  _main                           testcoop
```

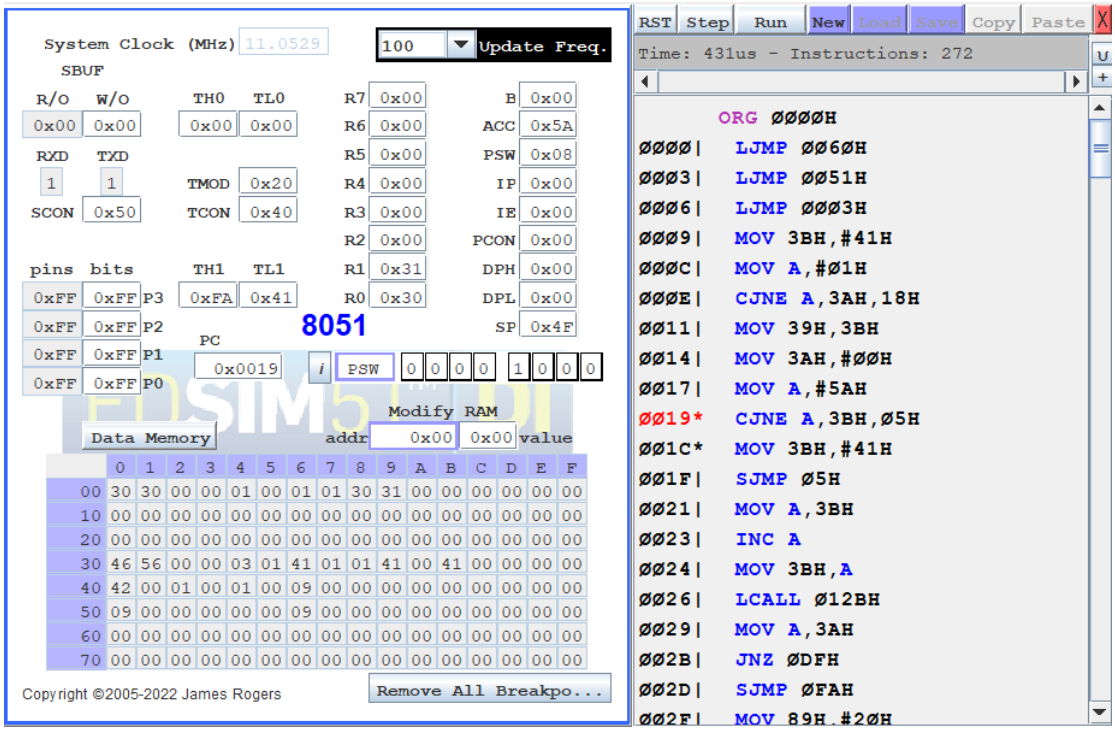
```
0000003A  _buffer_is_empty                    testcoop
```

```
00000038  _created_thread_ID                  cooperative
```

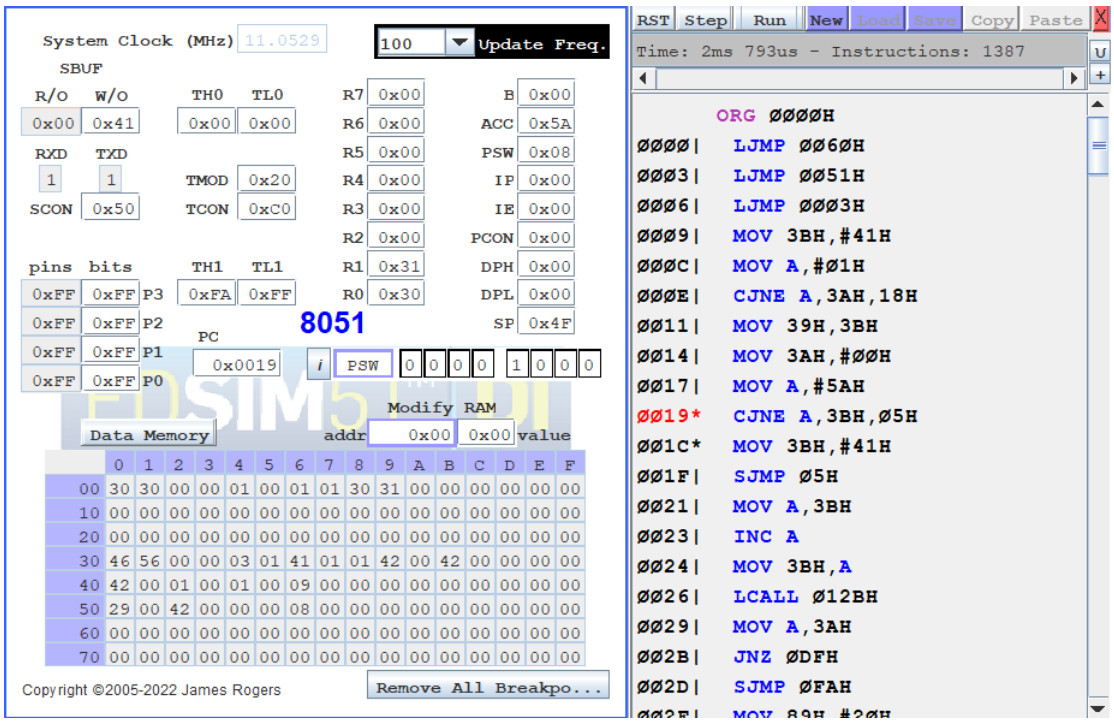
```c
void Producer(void) {
        /*
         * @@@ [2 pt]
         * initialize producer data structure, and then enter
         * an infinite loop (does not return)
         */
        __data __at (0x3B) char produced_character = 'A';

        while (1) {
                /* @@@ [6 pt]
                 * wait for the buffer to be available,
                 * and then write the new data into the buffer */
                if (buffer_is_empty == 1) {
                        shared_buffer = produced_character;
                        buffer_is_empty = 0;
                        if (produced_character == 'Z') produced_character = 'A';
                        else produced_character++;
                        ThreadYield();
                }
                while (buffer_is_empty == 0) {}
        }
}
```

Variable “produced_character” whose address is at 0x3B.

0x19: value of “produced_character”(at address 0x3B) is 0x41.



0x19: value of “produced_character”(at address 0x3B) gets changed to 0x42.

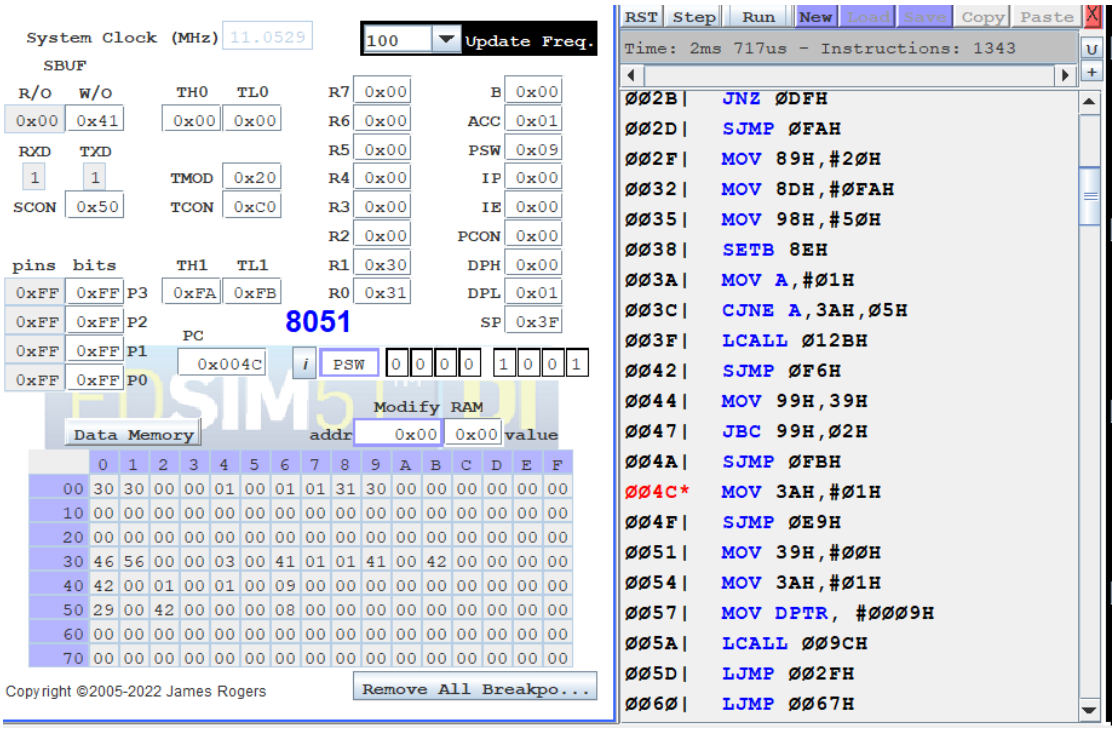


First, we could recognize that Producer() is currently running by the address of the function since the address of Producer() starts from 0x09. Second, we could check the value of 0x35 in memory, which is the ID of the currently running thread. We find

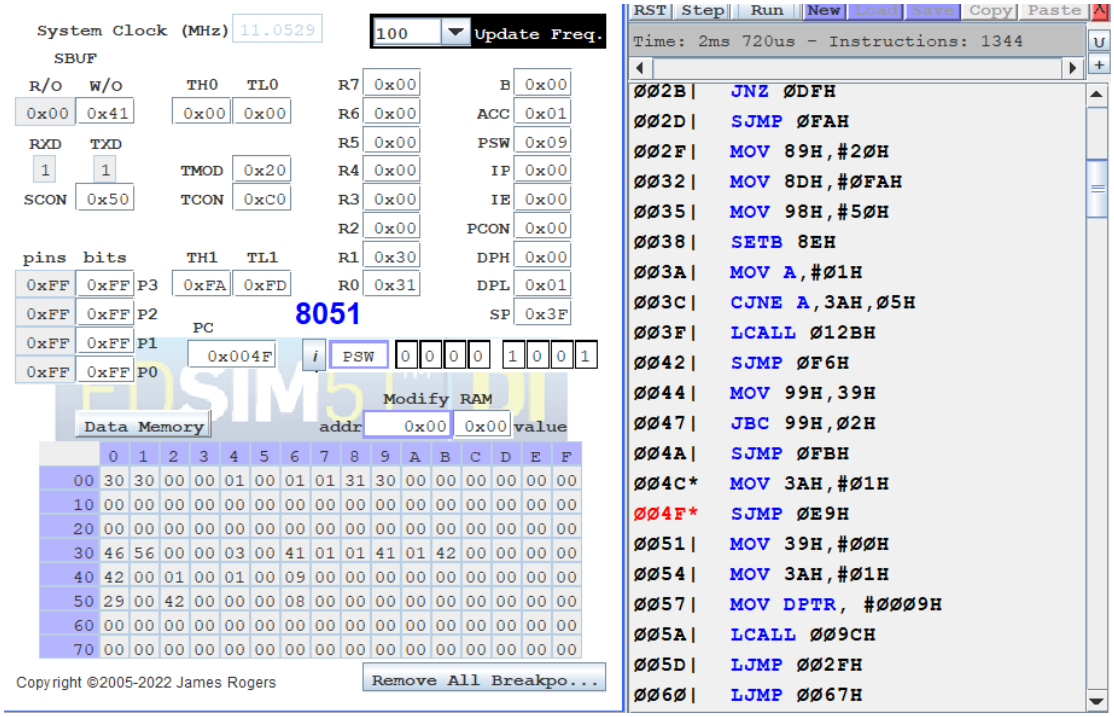
out that the currently running thread is 1, which is correct. Third, the value of the variable “produced_character” gets changed only in the function Producer(). So if we saw the value in memory 0x3B has changed, we could know that the current thread is running the Producer() function. For example, the screenshots above shows that the value of 0x3B has changed from 0x41 to 0x42, which means that function Producer() is running now.

3. We could know that Consumer() is running for several reasons:

0x4C: variable “buffer_is_empty” is 0



0x4F: variable “buffer_is_empty” gets changed to 1



First, we could recognize by the address of the function since Consumer starts from address 0x2F. Second, we could check the value of 0x35, which is the ID of the current running thread. We could find out that the current running thread is 0, which is correct. Third, the variable “buffer_is_empty” gets changed to 1 while the variable gets changed to 0 at the Producer() part. So when swapping between the two threads(Consumer and Producer), if we saw the value in address 0x3A gets changed from 0 to 1(the screenshots above), then we could know that the thread of Consumer() is running now.